

A Rule Embedding Method of Winrate Approximation for Texas Hold'em

Zhenzhen Hu

*College of Intelligence Science and Technology
National University of Defense and Technology
Changsha, China
hzzmail@163.com*

Weilin Yuan

*College of Intelligence Science and Technology
National University of Defense and Technology
Changsha, China
chensf005@163.com*

Mingwo Zou

*College of Intelligence Science and Technology
National University of Defense and Technology
Changsha, China
q2309008808@163.com*

Jing Chen

*College of Intelligence Science and Technology
National University of Defense and Technology
Changsha, China
chenjing001@vip.sina.com*

Shaofei Chen

*College of Intelligence Science and Technology
National University of Defense and Technology
Changsha, China
yuanweilin12@nudt.edu.cn*

Peng Li

*College of Intelligence Science and Technology
National University of Defense and Technology
Changsha, China
lipeng@nudt.edu.cn*

Jiaxing Chen

*College of Intelligence Science and Technology
National University of Defense and Technology
Changsha, China
chenjiaxingmtn@nudt.edu.cn*

Abstract—Expected winrate is one of the most important features in Texas Hold'em, and it is the foundation of state estimation and decision-making. Calculation of the expected winrate needs to consider the unknown opponent's hole cards and the undealt board cards. The huge combination space of these unknown cards makes the enumeration method time-consuming. A solution is using neural network to approximate the true winrate, but the calculation of the input features from cards information is fairly complex. In order to avoid feature engineering and improve the computing speed, a new neural network method based on rule embedding is proposed. With the elaborately designed convolution/max-pool kernel and a new proposed hypo-max-pool operation, the cards category rules are efficiently embedded into the winrate fitting network with several nodes. The result of experiments indicates that our rule embedding neural network method greatly outperforms the feature engineering method on the prediction accuracy and the computation efficiency.

Keywords—rule embedding, feature engineering, neural network, hypo-max pool, expected winrate

I. INTRODUCTION

Expected winrate is an important feature for decision-making in Texas Hold'em [1]. Players use it to evaluate states and choose actions. For example, when a player P observes

a raise action of his opponent O in two-player games, he can make a choice by comparing his expected winrate with the pot odds [2]. If the winrate is greater than the pot odds, choosing call action is reasonable because he may get future benefit in expectation with his investment of the call action. In addition, expected winrate is commonly used to construct the action probability model [3], [4] or to describe the style model [5] of the opponent. With these models and the observed information of the opponent, the hidden information of the opponent can be inferred. Players can benefit from these information, even perceive and exploit the weaknesses of the opponent, then obtain much more income [6]–[10].

Calculation of the expected winrate is a task of high computing complexity. If the player P is going to compute his expected winrate in two-player games, the undealt board cards and the opponent's hidden hole cards need to be enumerated. Through the enumeration process, the hand ranks of both sides are compared to computing the expected winrate, where the hand is namely the combination of the hole cards and the board cards, and the number of enumerations for Preflop, Flop, Turn and River round are $\binom{50}{2}\binom{48}{5} = 2097572400$, $\binom{47}{2}\binom{45}{2} = 1070190$, $\binom{46}{2}\binom{44}{1} = 45540$ and $\binom{45}{2} = 990$ respectively. Besides, if the player P wants to infer the hidden hole cards of the opponent O, the expected winrate of the op-

ponent O for every possible hole cards need to be considered, thus, all the above numbers should be multiplied by 990 [11]. Although the rank computation can be accelerated with some algorithms [12]–[16], the expected winrate computation with such large number of enumerations is still time-consuming.

Some approximation methods have been proposed to deal with this challenge. Monte-carlo method is used to reduce the number of enumerations [17]–[19], which can speed up the computation, but the enumeration is inevitable. The average rank strength (ARS) method [20] with several pre-computed tables for lookup can avoid the enumerations, and the computation is much faster, but more memory is needed to save and use the tables. The neural network approximation method can also avoid the enumerations by fitting the winrate directly, and it needs less memory than ARS method [21], but the input of the network is usually designed manually through feature engineering, and the calculation of the designed features is complicated and time-consuming. Therefore, it is necessary to find a method which can automatically learn effective features from the original data, so as to avoid feature engineering, and to establish a reasonable neural network with small memory requirements to fit winrate accurately.

In order to achieve the above target and inspired by the manner of Human learning and applying rules, we propose an idea of embedding rule knowledge to fitting task and setup a rule embedding neural network method in this work, the feature engineering is completely abandoned with an alternative rule embedding network, and the memory is limited by setting a suitable size of the neural network. Specifically, our proposals are as follows:

- use original data and rule embedding inspired by Human learning manner and embedding learning method to replace feature engineering;
- use a 5×14 2D-matrix cards combination representation, a new hypo-max pool operation, and several designed convolution/pool kernels to embedding the cards category rules;
- merge the rule embedding network into the winrate approximation network to improve the feature extraction and fitting performance.

We test our method with generated samples of different number of boards cards, and the average prediction error is compared with the feature engineering method, the result of experiments shows an obvious improvement on the computing speed and prediction accuracy.

II. BACK GROUND AND RELATED WORK

A. Texas Hold'em

Texas Hold'em is an incomplete information poker game, in which each player tries to win the chips betted by all players on condition of hidden private cards of the opponent. If the number of players is greater than two, we call the games as multi-player game, otherwise is the two-player game. Depending on whether the bet amount is limited or not, two types of the game: Heads-Up Limit Texas Hold'em and

Heads-Up No-Limit Texas Hold'em are defined [22]. Taking the Two-player Heads-Up No-Limit Texas Hold'em as an example, the repeated game is a match of individual games. In each individual game, there are four rounds: Preflop, Flop, Turn and River. In Preflop round, two hidden private cards (called as hole cards) are dealt to each player, the two players alternatively take actions of three types: Fold, Call/Check, and Bet/Raise depending on the action history, the pot information and his hole cards (specifically the expected winrate of his hole cards). The Call/Bet/Raise action means placing chips into the pot. In the next three rounds, three/one/one public cards are dealt on board (called as board cards), each player make decision according to the action history, the pot information, and the combination of the board cards and his hole cards (mainly the expected winrate of the combination). If the River round ends with no player previously folding to end the game, the outcome is determined by a Showdown. Each player reveals his two hole cards and the player that can form the strongest five-card combination wins all the chips in the pot. If both players have same strength combinations, the pot will be shared equally. The combination formed from the hole cards and the board cards is usually called as hand, and the strength of the hand is determined by hand rank. There are 7462 hand ranks for all the possible five-card combinations in the poker, Table I shows the hand rank examples.

TABLE I
POKER HAND RANK WITH EXAMPLES

Category	Example	sub-ranks
Straight Flush	As Ks Qs Js Ts	10
Four of a Kind	As Ah Ac Ad Ks	$\binom{13}{2} \binom{2}{1} = 156$
Full House	As Ah Kc Kd Ks	$\binom{13}{2} \binom{2}{1} = 156$
Flush	As Ks Qs Js 9s	$\binom{13}{5} - 10 = 1277$
Straight	As Kd Qs Jc Ts	10
Three of a Kind	As Ah Ac Kd Qs	$\binom{13}{3} \binom{3}{1} = 858$
Two pair	As Ah Kc Kd Qs	$\binom{13}{3} \binom{3}{2} = 858$
One pair	As Ah Kc Qd Js	$\binom{13}{4} \binom{4}{1} = 2860$
High Card	As Qh Jc Td 9s	$\binom{13}{5} - 10 = 1277$
total:		7462

B. Hand odds and expected winrate

Hand odds describes the winning probability at Showdown of a certain hand. People use varieties of metric to measure the Hand odds, such as Hand scores [23], Effective hand strength [24], Average rank strength [20], Expected winrate and so on. Unlike the hand rank evaluator is used to compute the rank of a given hand, the hand odds estimating tool is used to compute the probability of a certain hand being successful at the end of the game. So, the hand odds is usually computed and used by players to make decision before Showdown. Expected winrate as a winning rate in expectation can be computed with enumeration method considering the possible undealt board cards and the opponent's hidden hole cards. At Preflop round in the two-player game, the number of all the combinations of opponents' hole cards and the board cards are $\binom{50}{2} = 1225$ and $\binom{48}{5} = 1712304$ respectively, so the total enumeration

number is $1225 \times 1712304 = 2097572400$. By 2097572400 times of calculation and comparison of the hand rank of both sides, we can obtain the expected winrate by:

$$P_w = \frac{n_{ahead}}{n_{all}} + \frac{1}{2} \frac{n_{tied}}{n_{all}} \quad (1)$$

where, n_{ahead} is the number of enumeration cases in which our hand rank is greater than the opponent's hand rank, n_{tied} is the number of enumeration cases of both sides have same hand ranks, n_{all} is the number of all enumeration cases.

C. Common approximation method

Besides the direct computation with enumeration methods, there are several approximation ways to compute the expected winrate. The first is using Effective Hand Strength (EHS) to represent the winning probability, which is calculated by combining the Hand Strength with the Hand Potential [24], [25]. The Monte Carlo method uses smaller number of samples to replace the complete enumerations and can greatly reduces the computation amount, for example, in a competition with 2 opponents, the computation of winrate for the Flop round needs $P_{47,7} \approx 3.17 \times 10^{11}$ enumerations, but use $100 \times P_{45,4} \approx 3.58 \times 10^8$ samples to approximate, the error is about 0.012% [1]. In order to avoid the enumeration computation, Teofilo et al. [20] propose a table look-up method called Average Rank Strength (ARS), the Odds of hands are stored in a 10Mb table indexed by 7462 possible ranks of 5-card combinations and 169 IDs of pairs. Due to the many-to-one mapping from cards combinations to the cell value of the table, the Odds stored in the table is an average value of the mapped combinations. In addition, the Bayesian network is also used to model the probability distribution of Win, Lose and Tie given the current information of a hand [5], [26], [27], with the observable information in the game, the probability distribution can be inferred.

D. Neural network approximation method and feature learning

The neural network method is another approximation method which has lower computing complexity than the Monte Carlo methods and lower memory than the table look-up methods. Silva [21] uses a simple 29-24-12-2 neural network to approximate the probability of win and tie, which only takes up 8.4 KB of memory. After training with 250000 instances, the mean error of the test set is about 3%. Yan et al. [28] also use a 20-9-3 neural network to fit the level of hands. However, the input features of the networks are not learned automatically but designed by people according to their experience. The 29 features used by Silva include: the probability of forming all the hand types, the features of hole cards like card ranks or card suits, the features of board cards like category of the cards, and the round. Except the round information, all the others are empirical features, their design and selection may have great influence on the final fitting effect. In addition, most of the features need some calculation from the original cards information, which may

slow down the computing speed. In order to speed up the computation and improve the fitting performance, a possible solution is using the original information of cards instead of the human designed features as the input of the network, and then the network automatically learns the key features to fit the expected winrate. The proposed method of this work is to implement it.

III. METHODOLOGY

Our target is to build a new neural network method without feature engineering but with equivalent performance. Our idea is adding some knowledge to the neural network with original cards information input to ensure fine extraction of features of the cards. According to this idea, the original input and its transformation are investigated, and the knowledge—the human easily understood rules of the hand category—is found and embedded.

A. Description of rule embedding idea

In Texas Hold'em, the rank of each player's hand is compared to determine the final winner at showdown. The hand of each player is the combination of his hole cards and the communal board cards. For example, in a competition of two players, the player P holds two hole cards 'AsKd', his opponent O holds 'AcAd', and the board cards are '4s8dQdJsTc', at showdown the player P is the winner because his hand rank of the combination is 5863 with the best five cards 'AsKdQdJsTc' and the opponent's hand rank is 4082 with the best five cards 'AcAdQdJsTc', the former is a 'straight' and the latter is just an 'one pair'. The hand type categories is shown in Table I.

However, the situation is different before showdown, for example in flop round, the player P only know his hole cards 'AsKd' and the dealt 3 board cards '4s8dQd', so if he want to estimate his winrate, he has to consider the undealt 2 board cards and the opponent's possible hole cards in the rest $47(=52-2-3)$ cards, and the number of all the possible combinations is $\binom{47}{2} \binom{45}{2} = 1070190$. As we use the neural network approximation method other than the enumeration method, the available original information is only the five known cards 'AsKd' and '4s8dQd', so the network input x has to be captured from these known cards. At the training stage, the output y is given, and our target is training the network model $f(x)$ to minimize the loss $Loss(y, f(x))$. Due to the training samples are limited, we must ensure that the network has sufficient generalizing ability for the large space of cards combinations.

To sum up, we need to solve the following problems:

- use a suitable representation the original cards to feed the network;
- use a fine mechanism to make the network have sufficient generalizing ability

Figure 1 gives the idea of our solution, where we use a specific case as an example. For the first problem, we need to find a cards representation suitable for network feature learning to transfer the original cards into the network input:

$$x = Rp(h, b) \quad (2)$$

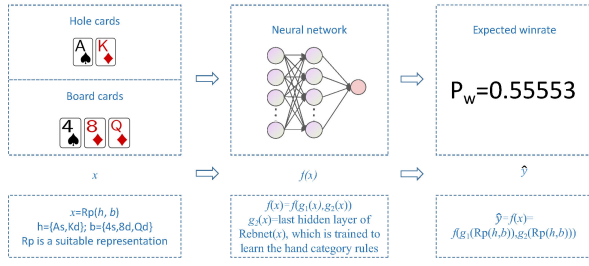


Fig. 1. The idea of rule embedding for winrate approximation. g_1 and g_2 are two parts of the whole network.

where, Rp is the representation function, h is the hole cards, b is the board cards, x is the network input.

For the second problem, our idea is find and use extra knowledge to improve the network's performance. Adding knowledge to neural network can improve training and prediction of the the network, which has been shown in some existing works [29], [30]. Not like the situation in [29], we have no expert knowledge on feature maps and introduce it by direct computation. However, we have a category table (Table I) that can be easily understood by human beings and human players usually use the rules in the table to evaluate the hand strength, so these rules can be introduced into neural network as knowledge, and the rest task is to find a way to introduce the rules. From the embedding learning perspective [31]–[36], the rules can be leaned as a dense low-dimensional representation. Thus, we can add the rule knowledge to the final network with a low-dimensional representation learned by embedding techniques.

Therefore, the input x flows into two parts of the network, the first part is any one type of traditional feature learning networks (like CNN, Multi-layer Dense Network) to extract some features $g_1(x)$. The other part is another feature extraction network based on rule embedding (or rule learning), which is used to intimate the Human's rule learning and applying manner and add extra knowledge to the final network. This part is critical to the generalization ability of the whole network, and it is pre-trained with the data set $D_R = \{(h_1, b_1, label_1), \dots\}$ implying the rules which can be easily observed and understood by Human beings. The input of the rule embedding part is the same as the whole network, and the last hidden layer of it is used as the extracted features $g_2(x)$.

As the last hidden layer of the rule embedding network is usually a dense layer, the features from the rule embedding part can be easily incorporated into the whole network, and the output of the whole network can be express as:

$$\hat{y} = f(g_1(x), g_2(x)) = f(g_1(Rp(h, b)), g_2(Rp(h, b))) \quad (3)$$

Therefore, based on suitable original cards representation and rule embedding, we establish a rule embedding network to achieve the winrate approximation.

B. Learning rules with embedding network

1) *Original input of the cards:* If we assign an integer to every card of the 52-card set, the original hole cards h and board cards b can be expressed as a vector with maximum length of 7 (where, 2 for h and 5 for b). But for a combination in a set, we usually use a one hot vector as the network input. Thus, as the size of the cards set is 52, we can use a 52-bit vector to represent the original cards information as the network input, where, each bit describes each card in the set, 1 represents present and 0 represents absent. Actually the winrate is not only related to the cards combination, but also depends on the hole cards. For example, the combination 'AsKd'+ '4s8dQd' is the same as '4s8d'+ 'QdAsKd' but their winrate are different because the hole cards are different ('AsKd' vs '4s8d'), so we can use a 104-bit vector to represent the card combination and the hole cards (the latter 52 bits describe the hole cards).

2) *Two dimensional transformation for rule embedding:* The one hot representation is an original input of the cards information without any additional knowledge. But if we want to add rule knowledge to the final network, we have to construct a suitable input for the rule embedding network. Observing the table I, we can find that the rules are intimately related to the rank and suit of each card in the combinations. The categories like One pair, Two pair, Three of a Kind, Full House and Four of a Kind depend on the number of equal rank cards, the categories like Straight and Straight Flush depend on the sequence of the cards' ranks, and categories like Flush and Straight Flush depend on the number of same suit cards. These category rules can be described through suitable operations like convolution on the rank and suit of the cards. So a two-dimensional matrix index by suit and rank transformed from the one-dimensional 52-bit is a better input for the rule embedding network. And for some special straight cases like 'As2s3s4s5s' and 'As2d3s4c5d', a size 5 kernel convolution can not capture the straight feature because the common rank sequence is '23456789TJQKA' and the distance between 'A' and '2' is larger than 5, so we insert a column same as the last column at the beginning and a row ignoring the suit at the end, Figure 2 shows the input 2D matrix for the combination of the board cards and the hole cards.

3) *Convolution and hypo-max pool operation:* The target of rule embedding is to obtain a dense low-dimensional representation from the sparse high-dimensional input. Several different convolution and pool operations are applied to implement it. We use three different two dimensional kernel convolutions to capture the equal rank, same suit and sequence features. The first 2D-Conv kernel is 4×1 with a stride 2×1 , the result is a 1×14 matrix. The second 2D-Conv kernel is 1×13 with a stride 1×2 , the result is a 5×1 matrix. The third 2D-Conv kernel is 1×5 , the result is a 5×10 matrix. By applying some max-pool operations to the result of covolutions, we can further extract the features. A 1×13 max-pool with stride 1×2 is used to obtain the max equal rank number feature, a 4×1 max-pool with stride 2×1 is

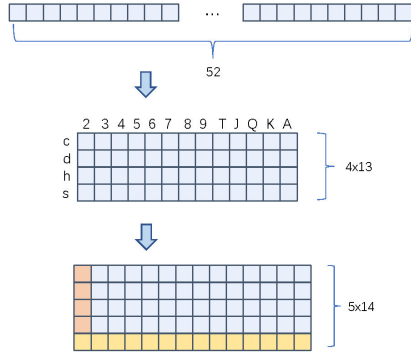


Fig. 2. Two dimensional representation of input. The brown column is the added column, the yellow row is the added row.

used to obtain the max same suit number feature, a 5×10 max-pool and 4×10 max-pool with stride 2×1 are used to obtain the max sequence length feature without or with same suit, Figure 3 shows the process.

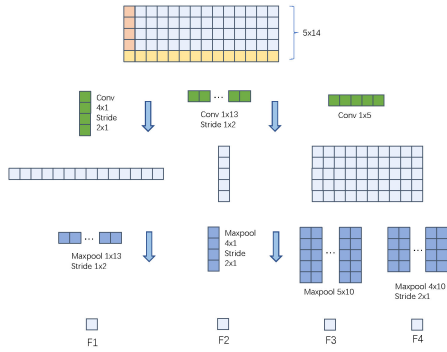


Fig. 3. Process of convolution and max-pool

However, for some categories like Full House, the maximum equal rank number feature is not enough, because the pair is also need to be captured. Thus, we propose a new pool operation called **hypo-max pool**, which can extract the second largest number of the input:

$$out = \text{hypo-max}(input) \quad (4)$$

For the matrix operation, the hypo-max-pool operation can be constructed with two max-pools and one unpool (unpool is an operation that computes a partial inverse of max-pool in which all non-maximal values are set to zero):

$$M_{out} = \text{hypo-max-pool}(M_{in}) \\ = \text{maxpool}(M_{in} - \text{unpool}(\text{maxpool}(M_{in}))) \quad (5)$$

Here, we use the hypo-max-pool operation other than the K-max pool operation [37], because we need not to obtain the order or structure information in this task. After applying three hypo-max-pools (1×13 , 4×1 , 5×10) to the result of the convolutions, we can obtain 3 extra features in addition to the 4 features obtained by max-pool, Figure 4 shows the process. With the max pool and hypo-max pool operation, the cards input is embedded as a vector with length of 7.

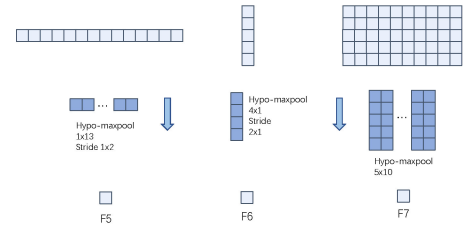


Fig. 4. Process of hypo-max-pool

4) *The rule embedding neural network:* After the feature extraction process, we use a full connection network to train the weight of the rule embedding representation, just append a output layer of 9 nodes for the 9 categories after the pooling process, Figure 5 shows the complete rule embedding network: RebNet.

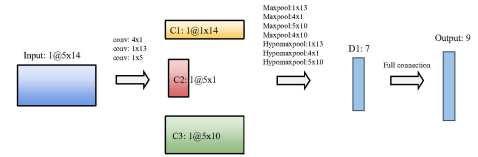


Fig. 5. Rule embedding network for training

According to Table I, we can generate enough samples to train it. After convergence, the hidden layers can be add to the winrate approximation network as the rule embedding part by preserving the weights.

C. Merge rules into the winrate approximation network

Our winrate approximation network is a multi-layer neural network (called WRNet) containing hidden layers of the RebNet, the input is the 2D-matrix representation of the known cards, and the output is the estimated winrate. In order to embedding the rules, the input is a 5×28 matrix, the last 14 columns are used to represent the hole cards information, the other 14 columns are used to represent the combination of the hole cards and the board cards (or only the board cards). The output is just a scalar in range $[0,1]$, so the output node is activated by the sigmoid function.

Two hidden dense layers (d1 and d2) are used. The connection between d1 and d2 (or between d2 and output) is the simple full connection. The connection between input and d1 has three components, the first component is a full connection from the two 4×13 parts to d1, the second component is the rule embedding network (r1) for the cards combinations, the 7 output nodes of r1 are fully connected with d1, the last component is another rule embedding network (r2) for the hole cards, which is constructed in the similar way as r1 but only use the max-pool operations to capture the equal rank, same suit and sequence features of two cards. The three output nodes of r2 are fully connected with d1. Where, the goal of using the full connection between r1/r2 and d1 is combining the features from embedded rules and the original representation.

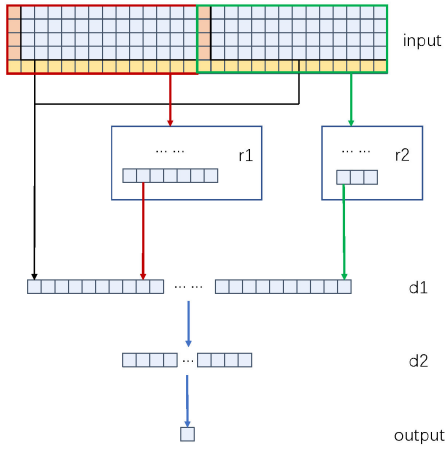


Fig. 6. Structure of the winrate network

Figure 6 gives the structure of the neural network WRNet, the network consists of two parts in general (the rule embedding part and the other), but because of considering the hole cards individually, the rule embedding part includes two components: $r1$ and $r2$.

IV. EXPERIMENTS

A. Experiment setup

All the experiments is conducted with pytorch, the rank of hand is calculated by Two-plus-two algorithm, and the winrate target for training is calculated with enumeration method. Experiments are classified into two parts, the first is the experiment for rule embedding, and the other is the experiment for winrate approximation. Time of computing winrate with the proposed neural network is compared with the neural network from [21], which is modified to adapt to pytorch. We use CrossEntropyLoss and MSELoss for training, and use the error rate (ER), absolute error (AE) and mean absolute error (MAE) on test dataset to measure the performance of hand category classification and winrate approximation for the neural network models $Y = \hat{f}(X)$:

$$ER = \frac{1}{N} \sum_{i=1}^N I(y_i \neq \hat{f}(x_i)) \quad (6)$$

$$MAE = \frac{1}{N} \sum_{i=1}^N L(y_i, \hat{f}(x_i)) \quad (7)$$

where, N is the size of data set, $I = 1$ if $y_i \neq \hat{f}(x_i)$ else 0, $L(y_i, \hat{f}(x_i)) = |y_i - \hat{f}(x_i)|$, $y_i - \hat{f}(x_i)$ is actually the AE of the i th instance from the data set, x_i is the input of i th instance to the networks, and y_i is the category label for the classification task, while y_i is the target winrate value for the winrate approximation task.

B. Neural network comparison for rule embedding

We constructed 4 different neural networks to investigate the effect of rule embedding:

- the RebNet proposed in section III-B4 with constant convolution kernels that all weights are set to 1;
- a full connection 52-7-9 network (called DSNet);
- a network similar to RebNet without the polling operations (5×14 input, the 3 convolutions proposed in section III-B3 and another size 1 kernel convolution, a 119 dense layer and 9-node output, called CNNet);
- a full connection 52-119-9 network (called DBNet)

We generate 4000 instances of 9 categories and the number of instances are set as 80, 400, 400, 800, 320, 400, 400, 800 for the 9 categories respectively to alleviate the class imbalance problem of the training data. We train the nets with batch size 1 and 3800/200 instances as training/test data. After 2000 epoches of training, the loss of training of the RebNet has dropped by 3 orders of magnitude (see the orange line in Figure 7), But DSNet and CNNet can not converge effectively (i.e. the red and green lines), DBNet converge to oscillation (i.e. the blue line). The RebNet's error rate ER of classification on test data is 0, which shows that the RebNet can capture the category rules well (see Figure 8(a)), and the ER of DSNet, CNNet, DBNet are 0.57, 0.435, 0.405 respectively (see Figure 8(b,c,d)), which shows that the rules have not been effectively learned by them. This result demonstrates the rule embedding capability of RebNet with a small number of training data in contrast to the huge number of 5/6/7 cards combinations.

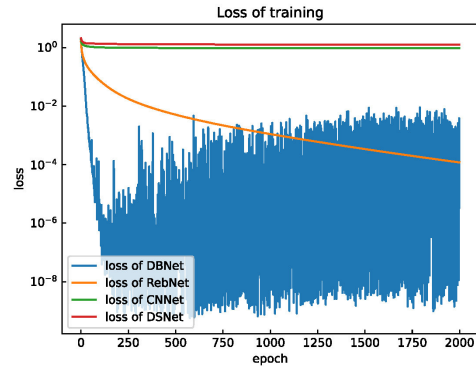


Fig. 7. Convergence curve of the 4 networks

From the weights between the dense layer and the output layer (see Figure 9, which is a contour graph of the weights, the horizontal 7 boxes are the feature set: [F1,F5,F2,F6,F3,F7,F4], the vertical 9 lines are the 9 categories from High card to Flush straight that are listed from top to bottom), we can find the relationships of the 7 features and the 9 categories as follows:

- F4 is dominant for Straight Flush;
- F1 is dominant for Four of a Kind and Three of a Kind;
- F5 is dominant for Full House, Two pair, High Card;
- F3 is dominant for Flush;
- F2 is dominant for straight;
- F7 is dominant for One pair.

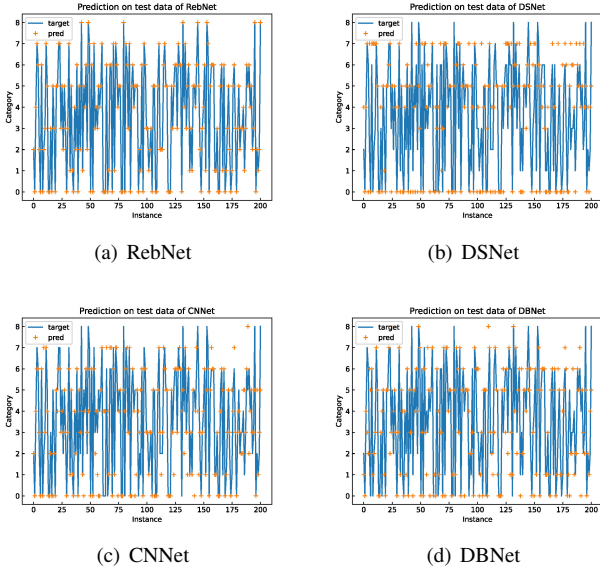


Fig. 8. Prediction of the 4 networks

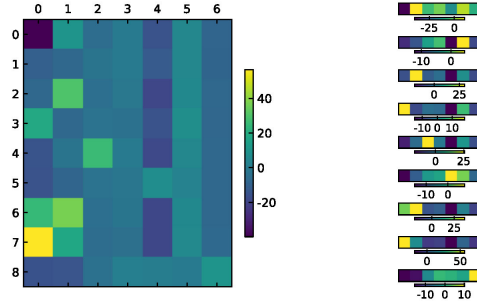


Fig. 9. Weights between D1 and output of RebNet. The left part is the weight contour for all the categories, the right part is the individual weight contour for each category.

These relationships shows that the important role of the hypo-max pooling operation of the rule embedding network, if we do not use it, the features of One pair, Full House, Two pair and High Card can not be captured effectively.

C. Performance of the winrate network with RebNet

We train the winrate approximation neural network (WRNet) with a data set of 200000 instances, 98 percent of them used as training data, and the rest is used as test data. The node number of layer d1 and d2 is set to 116 and 27. After 100 epoches of training with batch size 20 and learning rate 0.001, the loss of training drops to 10^{-3} (see Figure 10(a)). Figure 10 also shows the MAE history on test data, the AE history of two test instances, and the final absolute error of all

the test instances which are sorted by the real winrate value. After training, the final MAE on test data drops to 0.0143, and most of the instances have AEs in range $[-0.1, 0.1]$.

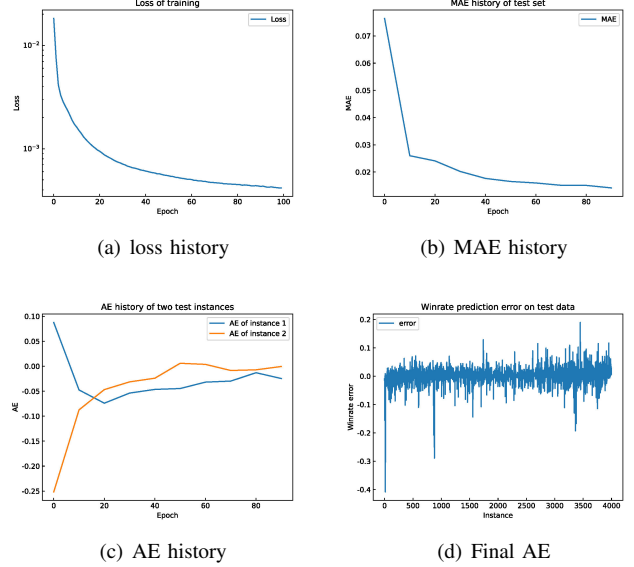


Fig. 10. Performance of the winrate network. The two instances in (c) is the first and second instance of the test data set.

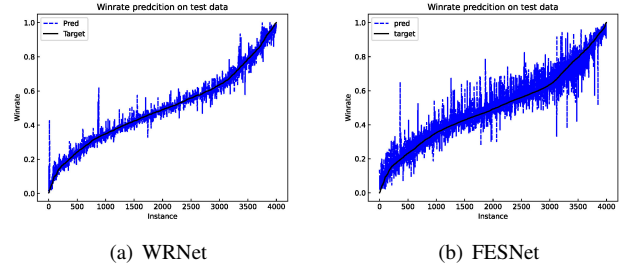


Fig. 11. Predictions of WRNet and FESNet

TABLE II
COMPARISON WITH FEATURE ENGINEERING METHOD

Deviation	WRNet	WRDnet	FESNet	FEBNet
	Ratio	Ratio	Ratio	Ratio
≤ 0.005	34.0%	27.6%	14.2%	14.4%
≤ 0.01	55.7%	46.1%	27.8%	27.8%
≤ 0.05	96.2%	91.8%	78.0%	78.6%
≤ 0.1	99.4%	98.1%	93.1%	93.4%
≤ 0.2	99.9%	99.6%	99.3%	99.4%
MAE	0.01430	0.02030	0.03420	0.03424

Figure 11 and Table II give the comparison with a multi-dense-layer network without rule embedding (called WRDnet) and the networks of the feature engineering method from [21]. The node number of hidden layers and training condition of WRDnet are same as WRNet. For the feature engineering method, we setup two neural networks to investigate the effect of the network scale: a small net (FESNet: 29-24-12-1) and

a big net (FEBNet: 29-116-27-1), they are all trained with 200000 instances with batch size 5 and 2000 epoches.

Figure 11 shows the prediction result of the test set for WRNet and FESNet, the bigger error of prediction winrate of FESNet can be observed easily. Table II shows MAE and test instances' Ratio comparison of the four networks, where Ratio means the ratio of test instances with the absolute value of AE are less than the Deviation to all the test instances. By comparing the result of WRNet and WRDnet, we can find the significant effect of rule embedding, with rule embedding the MAE reduces 29.6 percent. The result in the table also shows that WRNet outperforms the feature engineering method greatly, not only the small net FESNet, but also the enlarged big net FEBNet, the MAE almost drops by 58%, the Ratio of instances for each Deviation is obviously increased (for Deviation 0.05, the ratio of instances increases from 78.0% to 96.2%). The almost same result of FESNet and FEBNet indicates that the performance of the feature engineering method is limited by the feature engineering process and has little relationship to the network size.

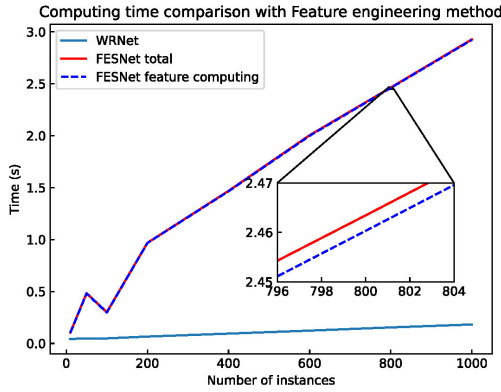


Fig. 12. Computing time of WRNet and FESNet

Figure 12 shows the average computing time comparison of the prediction with different number of instances, the computing time of WRNet is far less than the FESNet (for 1000 instances, the computing time of WRNet is 0.1821875 seconds, which is only 6.2% of the FESNet's 2.9278125 seconds), but the wights storage of WRNet is 71KB that is greater than the 6KB storage of the FESNet. For FESNet, the feature computing process almost takes up all of the computing time (see the almost overlapped red and blue lines in Figure 12, for the winrate computation of 1000 instances, the feature computing takes 99.85% of the total computing time), which indicates that feature computing is the bottleneck of the feature engineering method.

D. Influence of batch size and network scale on performance

For the purpose of reducing memory and improving the accuracy of prediction, we test the batch size of training and the cases of different number of nodes in d2 or d1. We first test the WRNet with different batch size to investigate the

effect of training batch size on the prediction accuracy of the test data set. Figure 13 shows the MAE comparison of batch size $= [1, 5, 10, 20, 40]$ with the number of epoches setting to 200. As the increase of the batch size, the MAE gradually decreases to a stable level, and the batch size 20 has the lowest MAE, which indicates that the batch size need to be set with a suitable value, neither too small nor too big.

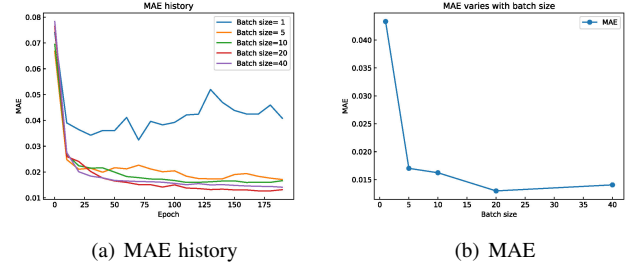


Fig. 13. Effect of training batch size

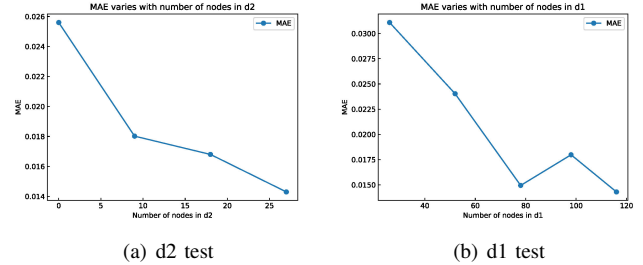


Fig. 14. Effect of network scale

WRNet is then tested with different number of nodes in dense layers. We first test the d2 layer with the number of nodes setting to 0,9,18,27 and node number of d1 is 116, then we test the d1 layer with the number of nodes setting to 26,52,78,98,116 and node number of d2 is 27, the number of training epoches for all of them is 100. The result of the MAE of these two tests is shown in Figure 14. With the increase of the network scale, the network has better performance in general, the network with node number of d1 and d2 are 116 and 27 has the lowest MAE of 0.01430. But note that as the network scale increases, more memory for the network is needed.

V. DISCUSSION

This study proposes a new rule embedding neural network method for winrate approximation to avoid the enumeration computation of all the possible card combinations. The rule embedding method is introduced to replace feature engineering, which can efficiently learn the rules of the hand category table with a small number of training instances in contrast to the huge number of cards combinations, particularly the proposed rule embedding network RebNet learns the rules of the space with millions of cards combinations only by 4000 instances. The elaborate convolution/max-pool kernel and the proposed hypo-max-pool operation make RebNet have

much higher performance than other networks. For the winrate approximation task, the rule embedding network plays important roles for the winrate approximation network, the mean absolute error of WRNet with rule embedding is obviously smaller than WRDnet without rule embedding, and WRNet also entirely outperforms the neural networks of the feature engineering method, not only upon the prediction accuracy, but also on the computing speed. The result of the experiments on batch size and network scale also tells us that the hyperparameters like batch size can be optimized. This study shows the power of the rule embedding method, and in our future research the embedding methods like rule/knowledge embedding will be applied to other aspects such as opponent modeling and decision-making in Texas hold'em.

REFERENCES

- [1] L. F. Teófilo, L. P. Reis, and H. L. Cardoso, "Computing card probabilities in texas hold'em," in *2013 8th Iberian Conference On Information Systems And Technologies (CISTI)*, 2013, pp. 1–6.
- [2] J. Rubin and I. Watson, "Computer poker: a review," *Artificial Intelligence*, vol. 175, no. 5-6, pp. 958–987, 2011.
- [3] J. Huang, "Building a computer poker agent with emphasis on opponent modeling," Master's thesis, Massachusetts Institute of Technology, Massachusetts, 2011.
- [4] X. Li, X. H. Jiang, Y. Z. Chen, and Y. J. Bao, "Game in multiplayer no-limit texas hold'em based on hands prediction," *Chinese Journal Of Computers*, vol. 41, no. 1, pp. 1–18, 2018.
- [5] R. B. Li, W. K. Li, L. Shang, Y. Gao, and M. J. Zhang, "Opponent's style modeling based on situations for bayesian poker," *Australasian Conference On Artificial Intelligence*, pp. 385–396, 2012.
- [6] S. Markovitch and R. Reger, "Learning and exploiting relative weaknesses of opponent agents," *Autonomous Agents and Multi-Agent Systems*, pp. 103–130, 2005.
- [7] A. Lazaric, M. Quaresimale, and M. Restelli, "On the usefulness of opponent modeling: the kuhn poker case study," *AAMAS*, pp. 1345–1348, 2008.
- [8] A. Ledezma, R. Aler, A. Sanchis, and D. Borrajo, "Ombo: An opponent modeling approach," *AI Commun.*, pp. 21–35, 2009.
- [9] S. Ganzfried and Q. Sun, "Bayesian opponent exploitation in imperfect-information games," in *2018 IEEE Conference on Computational Intelligence and Games (CIG)*, 2018, pp. 1–8.
- [10] S. Nashed and S. Zilberstein, "A survey on opponent modeling in adversarial domains," *Journal of Artificial Intelligence Research*, vol. 73, no. 5, pp. 1076–9757, 2022.
- [11] Z. Z. Hu, J. Chen, W. P. Zhang, S. F. Chen, W. L. Yuan, J. R. Luo, J. H. Xu, and X. Ji, "Odds estimating with opponent hand belief for texas hold'em poker agents," in *Artificial Intelligence. CICAI 2021. Lecture Notes In Computer Science, Vol 13069*. Cham: Springer International Publishing, 2021, pp. 51–64.
- [12] C. Kev. Cactus kev's poker hand evaluator. [Online]. Available: <http://suffe.cool/poker/evaluator.html>
- [13] Senzee5. Paul senzee on software. [Online]. Available: <http://www.paulsenzee.com/2006/06/some-perfect-hash.html>
- [14] Coding the wheel. The great poker hand evaluator roundup. [Online]. Available: <https://www.codingthewheel.com/archives/poker-hand-evaluator-roundup/>
- [15] Two Plus Two Newer Archives. Two plus two 7 card hand evaluators. [Online]. Available: <https://archives1.twoplustwo.com/showthread.php?t=288578>
- [16] H. Lee. Poker hand evaluator. [Online]. Available: <https://github.com/HenryRLee/PokerHandEvaluator>
- [17] XPokerEval. Xpokereval. [Online]. Available: <https://github.com/tangentforks/XPokerEval>
- [18] A. Prock. Pokerstove. [Online]. Available: <https://github.com/andrewprock/pokerstove>
- [19] A. Timo. Ompeval. [Online]. Available: <https://github.com/zekyll/OMPEval>
- [20] L. F. Teófilo, L. P. Reis, and H. L. Cardoso, "Estimating the odds for texas hold'em poker agents," in *Proceedings Of The 2013 IEEE/WIC/ACM International Joint Conferences On Web Intelligence (WI) And Intelligent Agent Technologies (IAT) On*, vol. 2, 2013, pp. 369–374.
- [21] B. Da Silva, "Approximating poker probabilities with deep learning," *Arxiv Preprint Arxiv:1808.07220*, 2018.
- [22] M. Moravik, M. Schmid, N. Burch, V. Lis, and M. Bowling, "Deepstack: Expert-level artificial intelligence in no-limit poker," *Science*, vol. 356, no. 6337, p. 508, 2017.
- [23] W. Chen and J. Ankenman, *The mathematics of poker*. Conjeleo, 2016.
- [24] D. Billings, D. Papp, J. Schaeffer, and D. Szafron, "Opponent modeling in poker," in *AAAI-98 Proceedings*. American Association of Artificial Intelligence, 1998, pp. 1–7.
- [25] D. Billings, A. Davidson, J. Schaeffer, and D. Szafron, "The challenge of poker," *Artificial Intelligence*, no. 134, pp. 201–240, 2002.
- [26] K. B. Korb, A. E. Nicholson, and N. Jitnah, "Bayesian poker," in *UAI'99 Proceedings Of The Fifteenth Conference On Uncertainty In Artificial Intelligence*, 1999, pp. 343–350.
- [27] K. Tretyakov and L. Kamm, "Modeling texas hold'em poker strategies with bayesian networks," tech. rep, techreport, 2009.
- [28] X. Yan, L. Xia, J. Yang, and Q. Zhao, "Opponent modeling in poker games," *Proceedings Of 2020 IEEE 9Th Data Driven Control And Learning Systems Conference (Ddcls'20)*, pp. 1090–1097, 2020.
- [29] H. Wang, "Renn: rule-embedded neural networks," *CoRR*, vol. abs/1801.09856, 2018. [Online]. Available: <http://arxiv.org/abs/1801.09856>
- [30] B. Chen, Z. Hao, X. Cai, R. Cai, W. Wen, J. Zhu, and G. Xie, "Embedding logic rules into recurrent neural networks," *IEEE Access*, vol. 7, pp. 14 938–14 946, 2019.
- [31] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, "Distributed representations of words and phrases and their compositionality," *CoRR*, vol. abs/1310.4546, 2013. [Online]. Available: <http://arxiv.org/abs/1310.4546>
- [32] P. Cui, X. Wang, J. Pei, and W. Zhu, "A survey on network embedding," *IEEE Transactions On Knowledge And Data Engineering*, vol. 31, no. 5, pp. 833–852, 2019.
- [33] F. Zhao, H. Sun, L. Jin, and H. Jin, "Structure-augmented knowledge graph embedding for sparse data with rule learning," *Computer Communications*, vol. 159, pp. 271–278, 2020. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0140366419319310>
- [34] P. G. Omran, K. Wang, and Z. Wang, "An embedding-based approach to rule learning in knowledge graphs," *IEEE Transactions on Knowledge and Data Engineering*, vol. 33, no. 4, pp. 1348–1359, 2021.
- [35] P. Bielak, K. Tagowski, M. Falkiewicz, T. Kajdanowicz, and N. V. Chawla, "Fildne: A framework for incremental learning of dynamic networks embeddings," *Knowledge-Based Systems*, vol. 236, p. 107453, 2022. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0950705121007152>
- [36] B. Dai, X. Shen, and J. Wang, "Embedding learning," *Journal of the American Statistical Association*, vol. 117, no. 537, pp. 307–319, 2022. [Online]. Available: <https://doi.org/10.1080/01621459.2020.1775614>
- [37] T. Zhou, X. Chang, H. Lu, X. Ye, Y. Liu, and X. Zheng, "Pooling operations in deep learning: From "invariable" to "variable"," *BioMed Research International*, pp. 1 – 17, 2022.